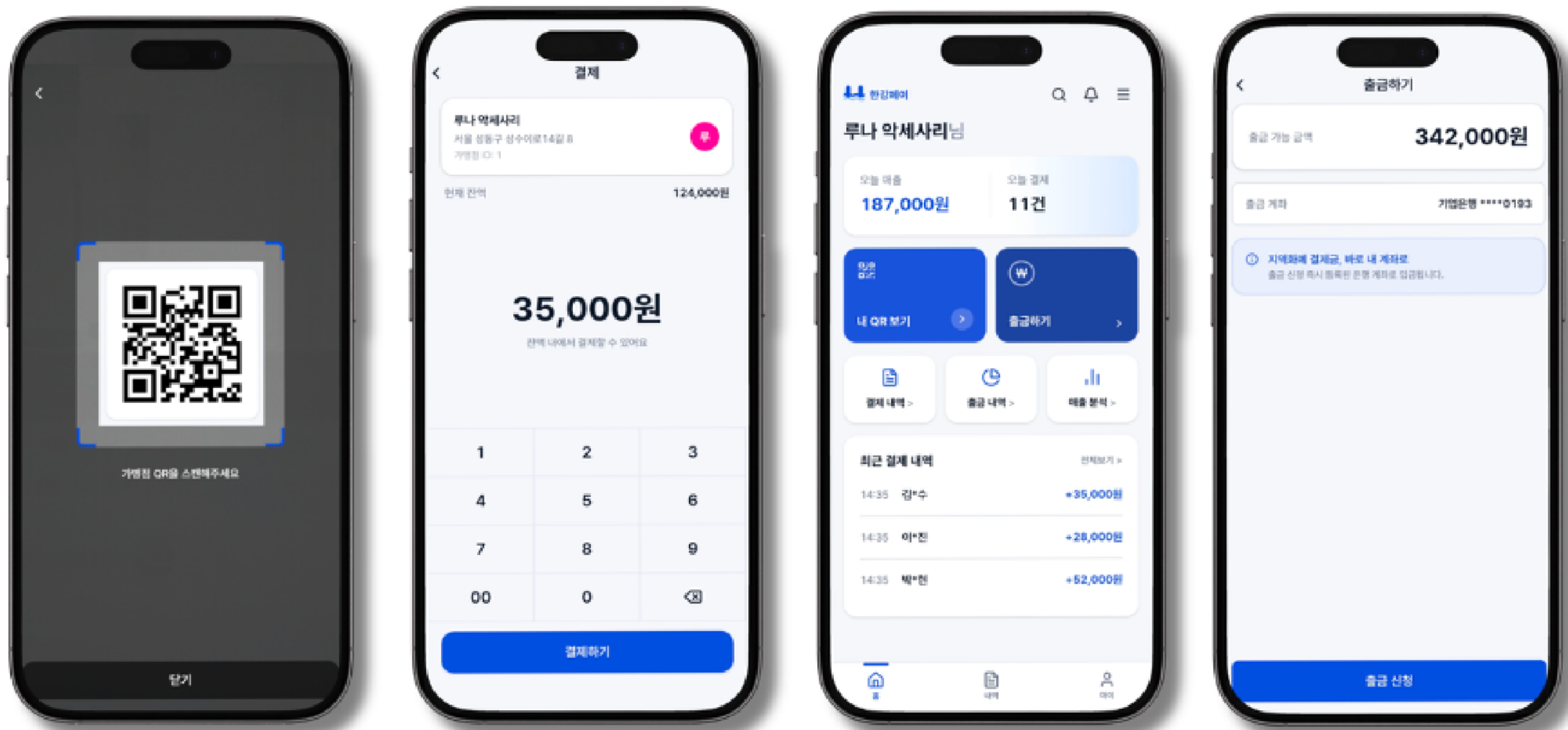


Project -

한강페이



서비스

<https://github.com/fluanceifi/hangang-pay/tree/dev>

블록체인 기반 지역화폐 결제 서비스

작업 기간

2026. 04. 23 - 2026.06. 17 (2개월)

백엔드 기술 스택

 Spring

 MySQL

 Redis

구성원

남인서 - PM & FE

유승준 - TL & BE

선지원 - BE & Infra

정가은 - BC

류경록 - BE

송준규 - Infra

역할

BackEnd

결제 트랜잭션 상태 설계

Redis 기반 멍등성 보장

Redis 기반 동시성 제어

Redis 기반 서버 간 부하 방지용 Rate Limit 설계

결제 실행 및 결제 취소 흐름 설계

분산 시스템 트랜잭션 경계 설계

분산 시스템 장애 복구 설계

클라우드 ↔ 온프레미스 간 VPN 터널링 구축

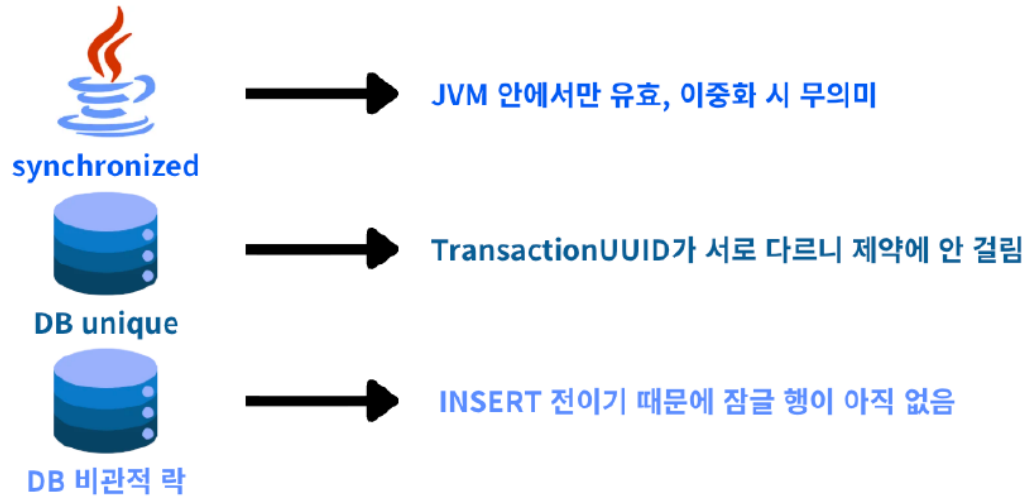
Sentry + Grafana Cloud 모니터링 통합

Trouble Shooting

소개 - 채널계 ↔ 계정계 ↔ 블록체인을 연동하는 결제 시스템을 설계하며 역등성/동시성/정합성/분산 트랜잭션/MQ 등 다양한 문제를 해결했습니다.

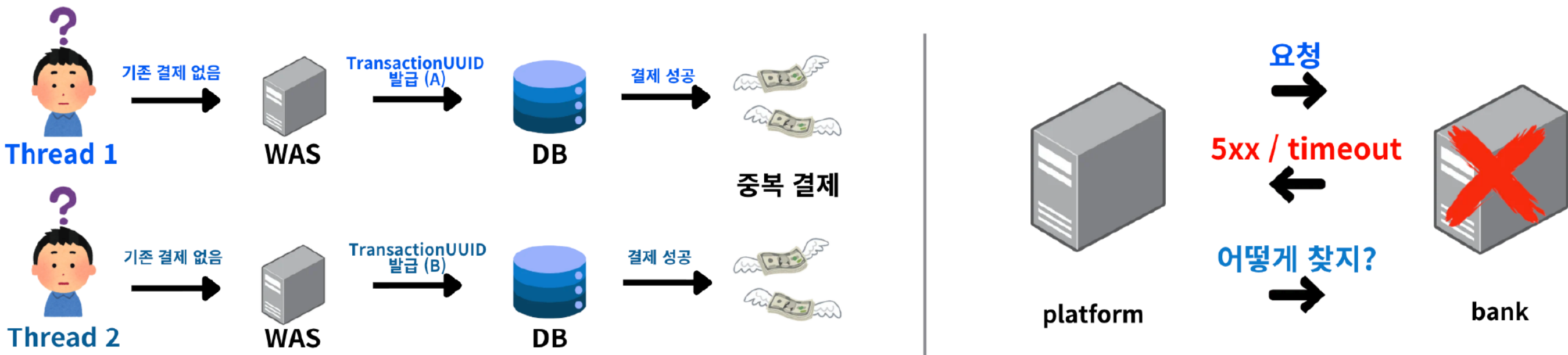
1. 같은 결제 요청이 항상 같은 락을 잡도록 설계한 중복 결제 방지

문제



- 결제 버튼을 연속으로 누르거나 네트워크 재전송이 발생하면 동일한 결제가 여러 건 생성됐습니다.
- 서버를 이중화하면 synchronized는 의미가 없습니다. 서로 다른 WAS의 JVM 락을 알 수 없기 때문입니다.
- 그래서 모든 서버가 공유하는 DB로 막으려 했지만, 너무 느리고 완벽히 막지 못했습니다.

원인 분석



- ① 막을 ‘대상’이 없었다.
 - 우측 세 가지가 전부 같은 이유로 실패했습니다. 결제 버튼을 두 번 누르면 서버가 TransactionUUID를 두 개 발급했습니다.
 - 중복 요청을 막는 모든 장치는 '같은 요청 인가?'를 판단합니다. 식별자가 매번 새로 생기면 경합 자체가 성립하지 않습니다.
- ② 은행에 물어볼 ‘식별자’가 없었다.
 - 식별자를 고정해 락이 동작하게 만든 뒤, 결제를 한 번의 요청으로 처리하니 다음 문제가 드러났습니다.
 - 락은 걸렸지만 그 안에서 외부 은행 서버를 호출해야 합니다. 여기서 타임아웃이나, Internal Server Error가 발생 시 은행이 처리했는지 알 수 없습니다.
 - 확인 하려면 은행에 '이 결제 처리했나?'고 물어봐야 합니다.
 - 그런데 TransactionUUID는 요청을 받은 뒤 서버 메모리에서 만들어졌을 뿐, 은행에 보내기 전 어디에도 기록하지 않아, 물어볼 식별자가 없었습니다.

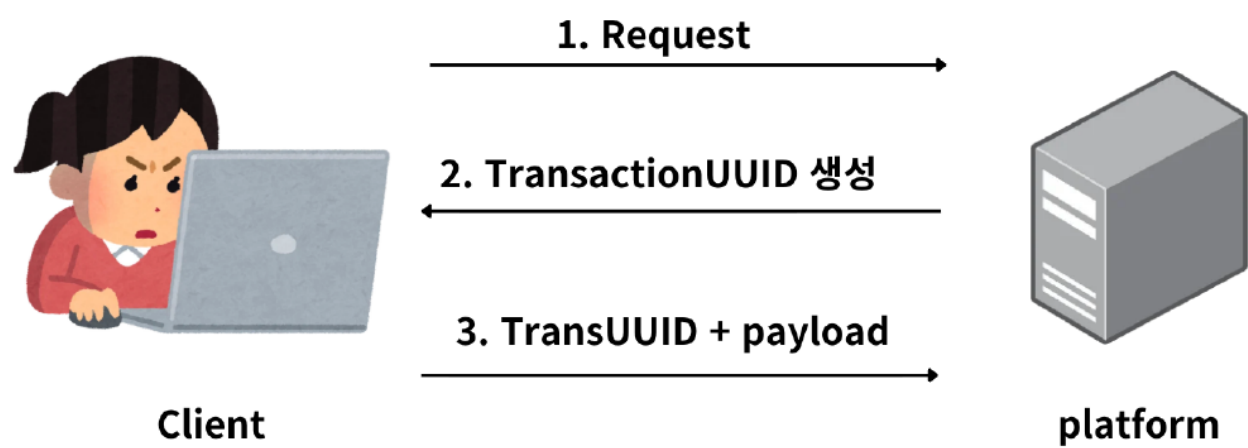
해결

① 모든 서버가 공유하는 락 - Redis (Lettuce / Redisson)

구분	Lettuce	Redisson
락 대기	스핀 (계속 재시도)	Pub/Sub 알림
대기 중 Redis 부하	있음	거의 없음
의존성	추가 없음	추가

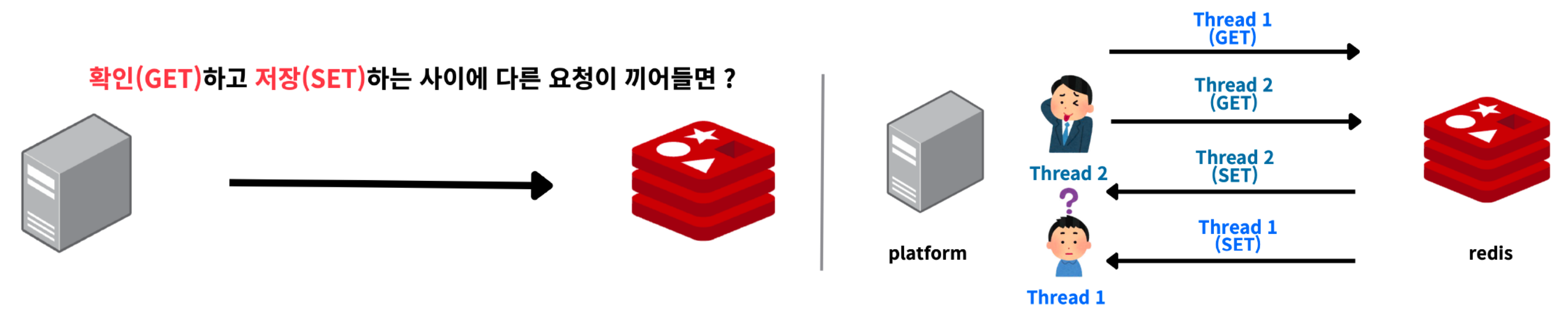
- **Redisson** 장점은 ‘기다릴 때’ 입니다. 하지만 결제는 기다리면 안 됩니다. 이미 실행 중인 결제는 **409로 차단해야** 합니다.
- 대기 자체가 없으니 Redisson이 풀어주는 문제도 생기지 않습니다. 따라서 SET NX 한 줄로 충분했고, 의존성을 추가하지 않았습니다.

② 결제를 2단계로 분리 (기록 후 실행)



- 실행 전에 ‘결제 시도’를 먼저 만듭니다. 서버가 GTID를 발급해 DB에 저장하고, 클라이언트는 그 GTID를 실행 요청으로 되돌려 보냅니다.
- 이러면 결제 버튼과 식별자 생성이 분리됩니다. 몇 번 눌러도 GTID는 하나입니다. 자연스럽게 ‘원인②’도 해결됩니다.
- 은행에 보내기 전에 이름이 DB에 기록돼 있으니, 거래가 유실되도, GTID로 은행에 조회해 결과를 확인할 수 있습니다. 자세한 내용은 뒤에서 다룹니다.

③ 확인과 저장을 하나의 명령으로 처리 (SET NX)



```
Boolean acquired = redisTemplate.opsForValue().setIfAbsent(key, token, LOCK_TTL); // SET NX

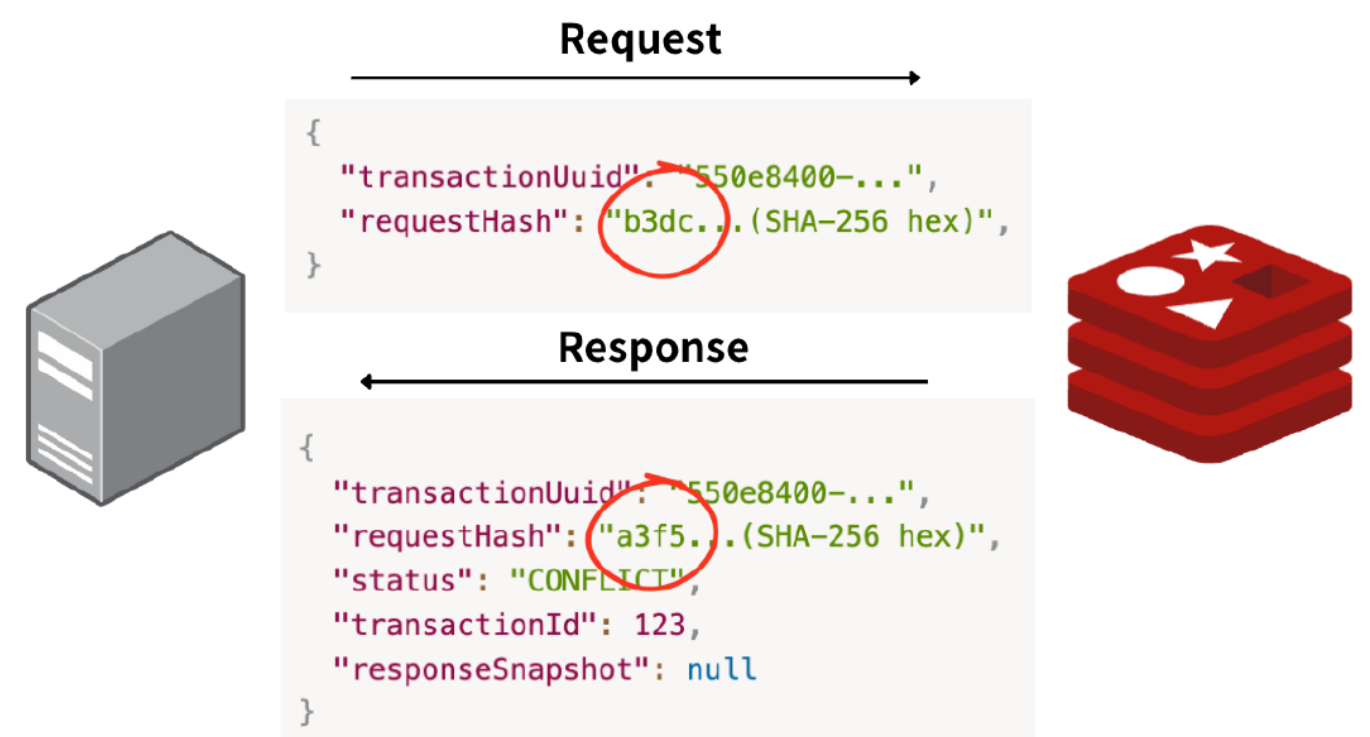
if (!Boolean.TRUE.equals(acquired)) throw new BusinessException(PAYMENT_ALREADY_PROCESSING); // 409, 재시도 없음
```

- **GET(확인)과 SET(저장)이 분리되어있으면** 두 명령 사이에 다른 요청이 끼어듭니다. 따라서 **setIfAbsent**로 단일 명령 처리했습니다.
- Redis는 명령을 단일 스레드로 순차 처리하므로 끼어들 틈이 없고, 선점에 성공한 하나만 실행합니다.

④ 클라이언트가 조작할 수 없는 맥등키 (requestHash)

- 먹등 판정 해시를 클라이언트가 보낸 값이 아니라 **DB에 저장된 결제 시도의 값**(보낸 사람, 받는 사람, 금액) 으로 계산합니다.

상태	조건	처리
NEW_REQUEST	없음	처음 온 요청 (PROCESSING 선택)
CONFLICT	같은 key + 다른 Value	같은 UUID + 다른 요청내용
RETURN_SNAPSHOT	같은 key + 같은 Value + 완료	이미 완료된 동일 요청
ALREADY_FAILED	같은 key + 같은 Value + 실패	복구를 실패한 거래 → 재시도 거절
PROCESSING	같은 key + 같은 Value	먼저 온 요청이 아직 처리중



- 클라이언트가 실행 요청의 금액을 바꿔 보내도 해시 입력이 바뀌지 않으므로 해시 자체를 조작할 수 없습니다.
- 저장된 해시와 다르면 **409 IDEMPOTENCY_CONFLICT** 입니다.
- 금액은 **BigDecimal.stripTrailingZeros()** 로 정규화해 1.0과 1.00이 다른 결제로 갈리지 않게 했습니다.

결과

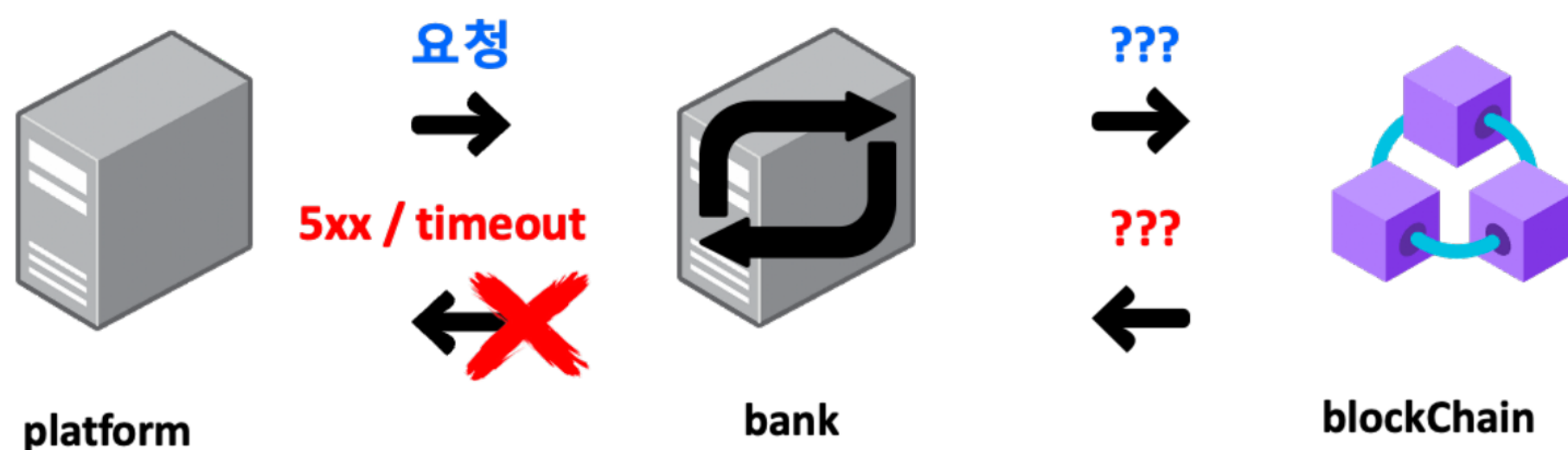
- K6 기반 부하테스트 환경에서 VU 10을 기준으로 역등성 테스트를 진행하였습니다.
 - 성공 : 1건
 - 중복 요청 : 9건(409)
 - 중복 결제 : 0건
- 동일 결제가 한 번만 생성됨을 검증하였습니다.

- ✓ payment intent 성공
- ✓ payment burst: 처리 식별자가 정확히 1개
- ✓ payment burst: 2xx/409 외 응답 없음
- ✓ payment retry: 동일 스냅샷 반환

CUSTOM

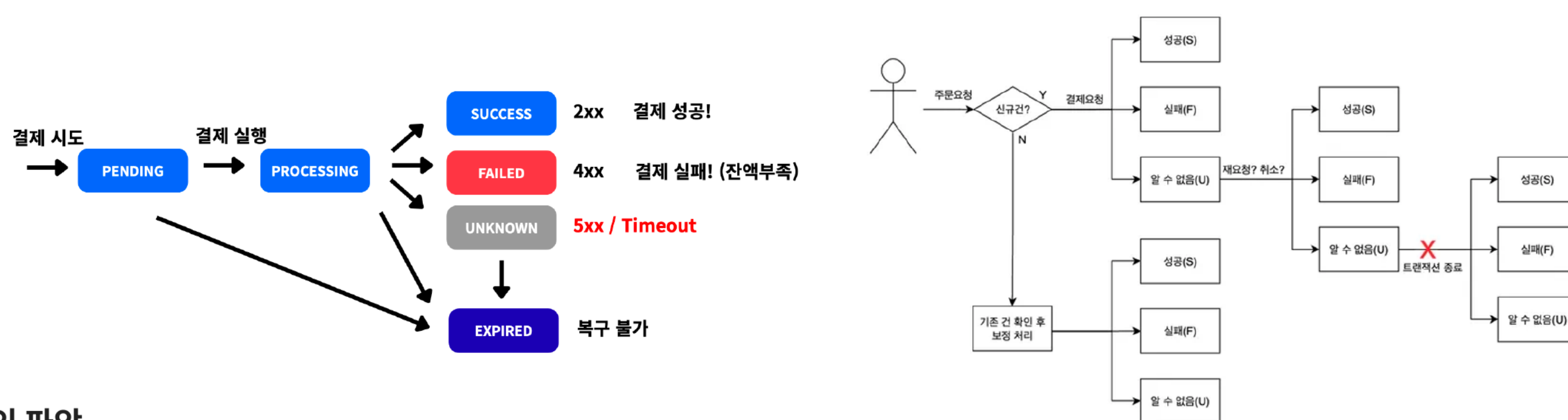
idempotency_burst_2xx.....	1
idempotency_burst_blocked_409.....	9
idempotency_executed_once.....	1
idempotency_retry_snapshot_returned...	1
idempotency_violation.....	0

2. 분산 시스템에서 장애를 대처하는 법: 결제 복구



문제

- 블록체인을 이용한 결제 시스템은 플랫폼 → 은행 → 블록체인 구조로, 은행 DB의 결제 내역을 기반으로 분산 원장에 결제를 기록합니다.
- 은행 호출 중 HTTP 5xx 코드, Timeout 문제가 발생하면 성공인지 실패인지 알 수 없습니다.
- 이걸 단순히 “실패”로 처리했더니, 돈은 빠져나갔지만 사용자에게 실패로 안내되는 정합성 문제가 생겼습니다.
- 정합성을 맞추기 위해 복구 트랜잭션을 추가하였지만, 서버가 다운되는 문제가 발생하였습니다.



원인 파악

- 분산 시스템은 하나의 서버가 아니기 때문에 외부 API를 호출 할때 트랜잭션을 분리시켜야합니다.
- 때문에 결제가 지금 어디까지 진행됐는지, 다음에 무엇을 해야 하는지 서버가 명확하게 알고 있지 못하고 있었습니다.
- 복구를 시도해도 실패한 복구 트랜잭션에 대해 계속 재시도하는 문제가 존재하여 무한 루프에 빠지는 일이 있었습니다.

해결

- 서버가 상태를 알 수 있도록 성공 / 실패 외에도 **결제 시도**, **처리 중**, **알 수 없음**, **복구 불가**에 대한 상태를 설계했습니다.
- 애매한 건은 곧장 실패로 단정하지 않고, 다시 시도해볼 만한 경우만 딱 한 번 재시도하여 무한루프를 방지하였습니다.
- 그래도 여전히 "**알 수 없음**" 상태라면, 잠시 보류했다가 스케줄러를 이용해 지수 백오프 방식으로 10회 시도합니다.
- 실패한 복구에 대해서는 "**복구 불가**" 상태로 변경합니다.

결과

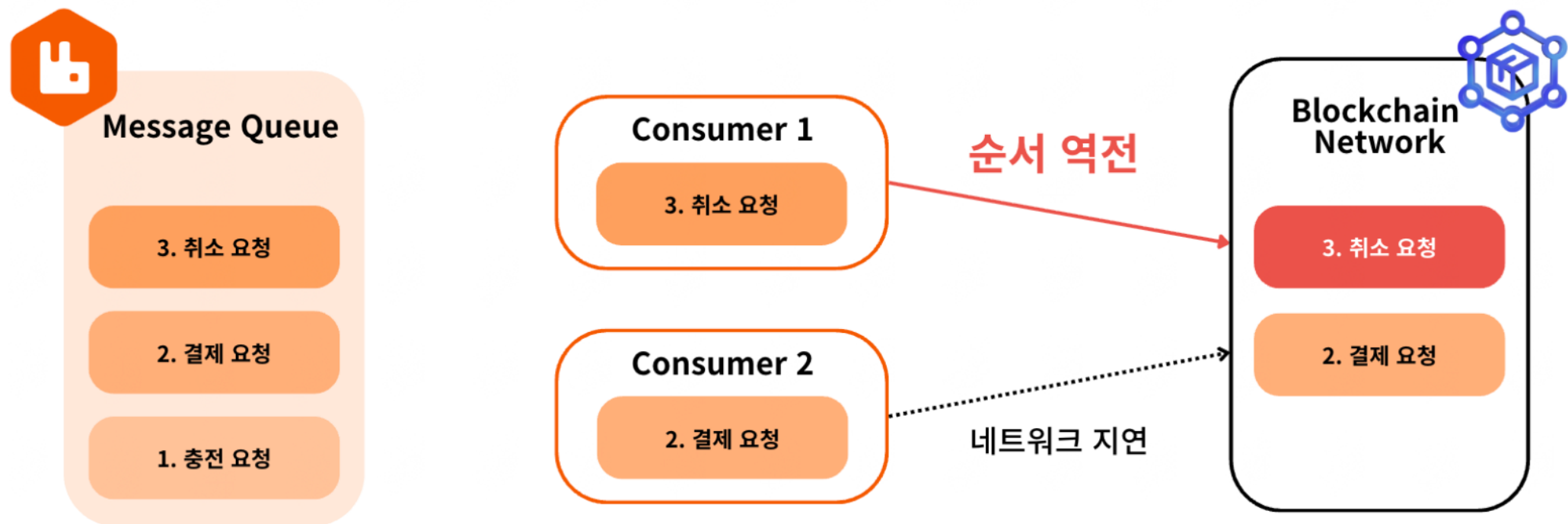
- 스케줄러를 이용하여 **사용자의 화면 이탈**, **타임아웃**, **서버 다운** 같은 애매한 상황들을 **고아상태**로 남기지 않고 전부 자동으로 복구됐다.
- 심사위원에게 **"알수 없음"** 상태를 다룬 건 지금까지 이 팀이 처음이라는 평가를 받았다.

```
2026-06-14T20:59:00.584+09:00 INFO 64107 --- [hangang-pay] [ scheduling-1] []
.h.d.t.s.UnknownPaymentRecoveryScheduler : 결제 복구 스케줄러 실행 : 대상 =68
```

3. 아웃박스 패턴으로 안전하게 비동기화하고 처리 순서 보장하기

한 줄 소개

블록체인 연동을 아웃박스 패턴으로 비동기화해 스레드 고갈과 메시지 유실을 막았고, 그 파이프라인 위에서 발생한 순서 역전 문제를 해결했습니다.

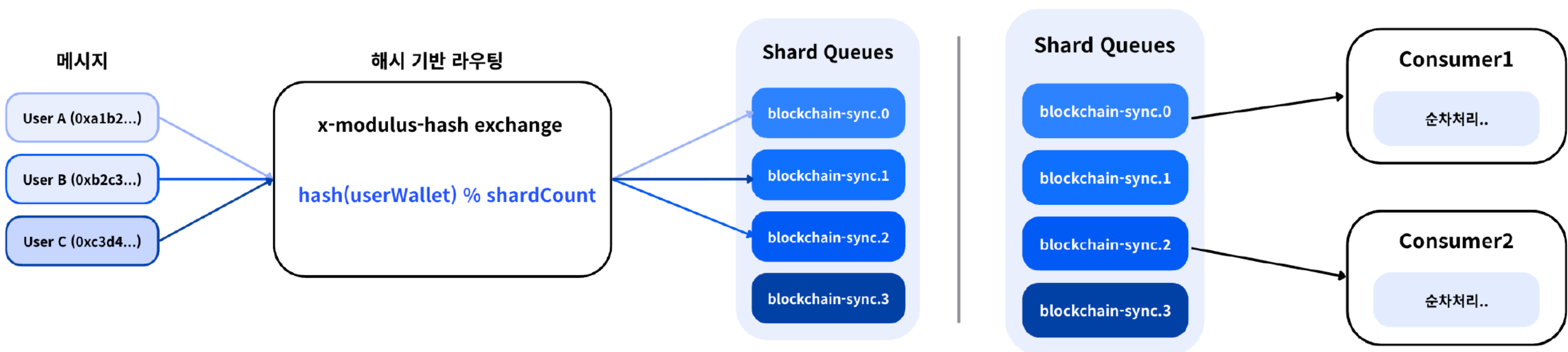


문제

- 블록체인은 트랜잭션 확정까지 수 초가 걸립니다. 동기로 기다리면 서버 스레드가 점유된 채 대기해 응답 지연과 스레드 고갈이 발생합니다.
- ‘DB 거래 기록 저장 + MQ에 메시지 발행’을 분리하면, 둘 중 하나만 성공했을 때 Message Loss(거래는 됐는데 블록체인 미반영) 또는 Ghost Message(롤백됐는데 발행됨)로 원장과 블록체인이 어긋납니다.
- 또한 여러 컨슈머가 병렬로 메시지를 처리하면, 같은 사용자의 결제 → 취소가 뒤바뀌어 취소가 먼저 반영될 수 있습니다.

원인 파악

- **이중 쓰기:** DB 트랜잭션과 MQ 발행은 서로 다른 시스템이라 하나의 원자적 커밋으로 묶을 수 없습니다.
- **순서 역전:** MQ는 큐가 하나여도 컨슈머가 여러이면 처리 순서를 보장하지 않고, 네트워크 지연으로 뒤 메시지가 먼저 도착할 수 있습니다.
- **기술 특수성:** 블록체인은 지갑 계정마다 트랜잭션 순서를 나타내는 nonce가 있어, 한 계정의 트랜잭션을 0 → 1 → 2 순서대로 처리해야 합니다.
 - 기획 단계에서 '발행 기관이 지역화폐를 발행/이전한다'라고 설계했기 때문에 은행 계정을 1개만 발급했습니다. (Single signer)
 - 때문에 여러 컨슈머가 동시에 처리하면 nonce가 겹치거나 순서가 뒤섞여 일부 트랜잭션이 실패할 수 있습니다. (Race Condition)
 - 처음엔 '단일 큐 + 컨슈머 스레드 1개'로 전부 직렬 처리했지만, 부하가 늘어도 대응할 수가 없었습니다.



해결

- **아웃박스로 이중 쓰기 차단:** 요청 스레드가 하나의 DB 트랜잭션에서 은행 원장과 아웃박스를 함께 기록해 원자적으로 커밋했습니다.
 - 별도 스케줄러가 5초 주기로 아웃박스를 Polling → MQ로 발행, 실패 시 지수 백오프 방식으로 3회 재시도 후, 최후엔 DLQ로 보냅니다.
 - 커밋과 발행이 분리돼도 유실 없이 최종 발행을 보장하도록 했습니다. (At-Least-Once 보장 방식)
- ① **순서 보장:** 순번 발급·선행 검증
 - DB에서 사용자 지갑 주소별로 비관적 락을 걸어 seq_no를 원자적으로 발급.
 - 스케줄러가 선행 seq의 처리 여부를 확인해 성공하기 전에는 후 작업의 seq를 발행하지 않도록 제한.
- ② **순서 보장:** 라우팅·소비
 - RabbitMQ x-modulus-hash exchange로 단일 큐를 사용자 지갑 해시 기준 4개 샤드 큐로 분할하여 같은 사용자는 항상 같은 샤드 큐로 전달.
 - 각 샤드 큐마다 Single Active Consumer(concurrency=1)를 뒤서 큐마다 활성 컨슈머를 하나로 고정하여 큐에 쌓인 작업은 직렬화 처리 (FIFO)
 - 덕분에 같은 계정의 트랜잭션이 동시에 서명·제출되지 않아 앞서 말한 nonce 충돌 해결

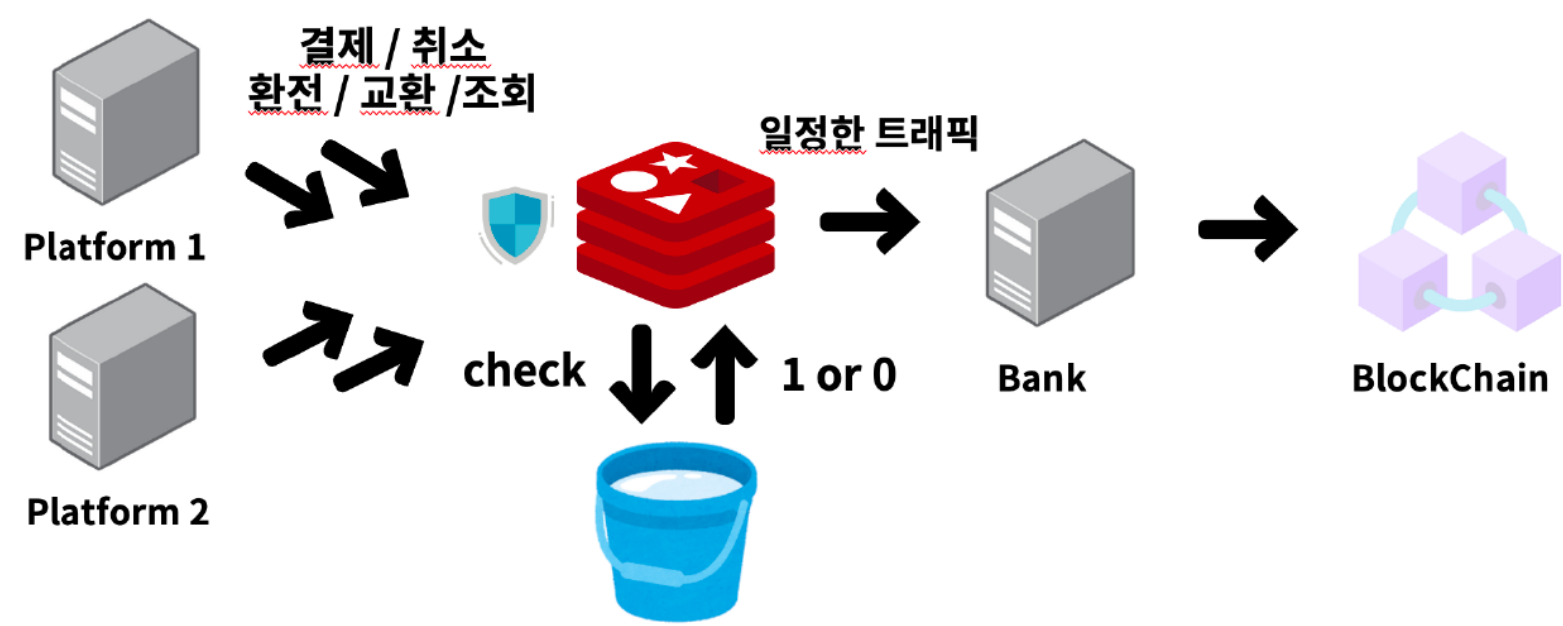
```
QueueBuilder.durable(queueName).singleActiveConsumer()...

@RabbitListener(queues = "#{@blockchainSyncShardQueueNames}", concurrency = "1")
```

결과

- 서버 스레드를 블록체인 대기에서 해방해, 한정된 자원으로 **응답 시간을 유지하며 처리량을 확보**했습니다.
- DB 커밋과 MQ 발행이 분리돼도 메시지 유실 없이 반영되어, **원장과 블록체인의 정합성**을 보장했습니다.
- 같은 사용자의 거래는 반드시 **발생 순서대로** 반영되면서 샤드 단위 병렬성은 유지하고 **순서와 처리량을 동시에 확보**했습니다.

3. Rate Limit을 활용한 대용량 트래픽 방어와 부하 테스트 병목 분석



문제

- 결제뿐만 아니라 플랫폼에서 호출하는 결제 / 취소 / 충전 / 환전 / 조회 API는 자체 구축한 1대 뿐인 은행 서버에 모든 요청을 부과합니다.
- K6 환경에서 VU300 기준, 한 가맹점에 결제를 몰아넣으니 TPS는 초당 47에서 막히고, 통과한 요청도 응답까지 약 5.33초가 걸렸습니다.
- 같은 결제를 “즉시 거절” 하도록 만들어 놓음에도, 실제로는 그 거절(429)조차 똑같이 5.33초가 걸렸습니다.



원인 파악

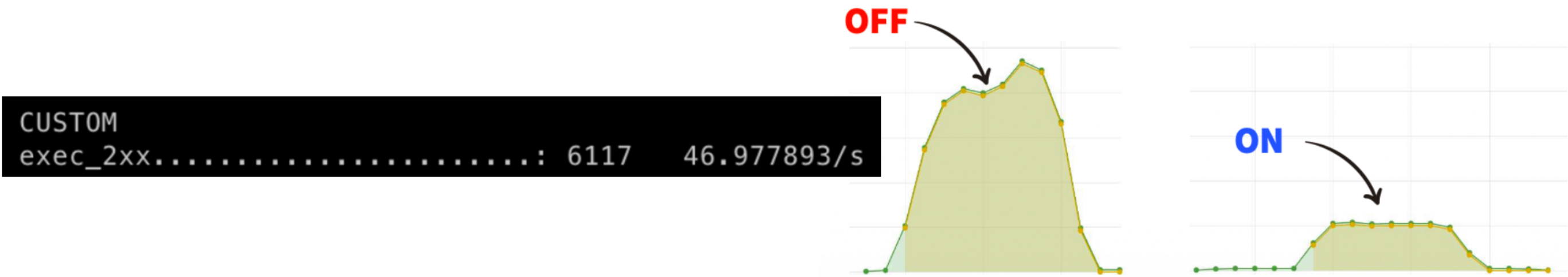
- Redis Token Bucket을 사용하자 차단 자체는 잘 됐습니다. → 초과 요청이 초당 16건씩 차단
- 병목이 CPU에 존재하지 않는 다는걸 확인 후, Rate Limiting ON/OFF 측정 → 요청의 지연이 똑같다는 걸 파악했습니다.
- 모든 결제가 하나의 가맹점에 몰리기 때문에 은행 DB에 row-lock이 걸린 후 직렬화 + 커넥션 풀 10개 → 스레드당 5초 대기
 - 플랫폼 서버에서는 동기 블로킹 호출(RestClient)을 이용하여 그 5초를 고스란히 대기합니다.
- 이런식으로 후속 요청들이 점점 스레드 큐에 쌓여 대기하였습니다.

해결

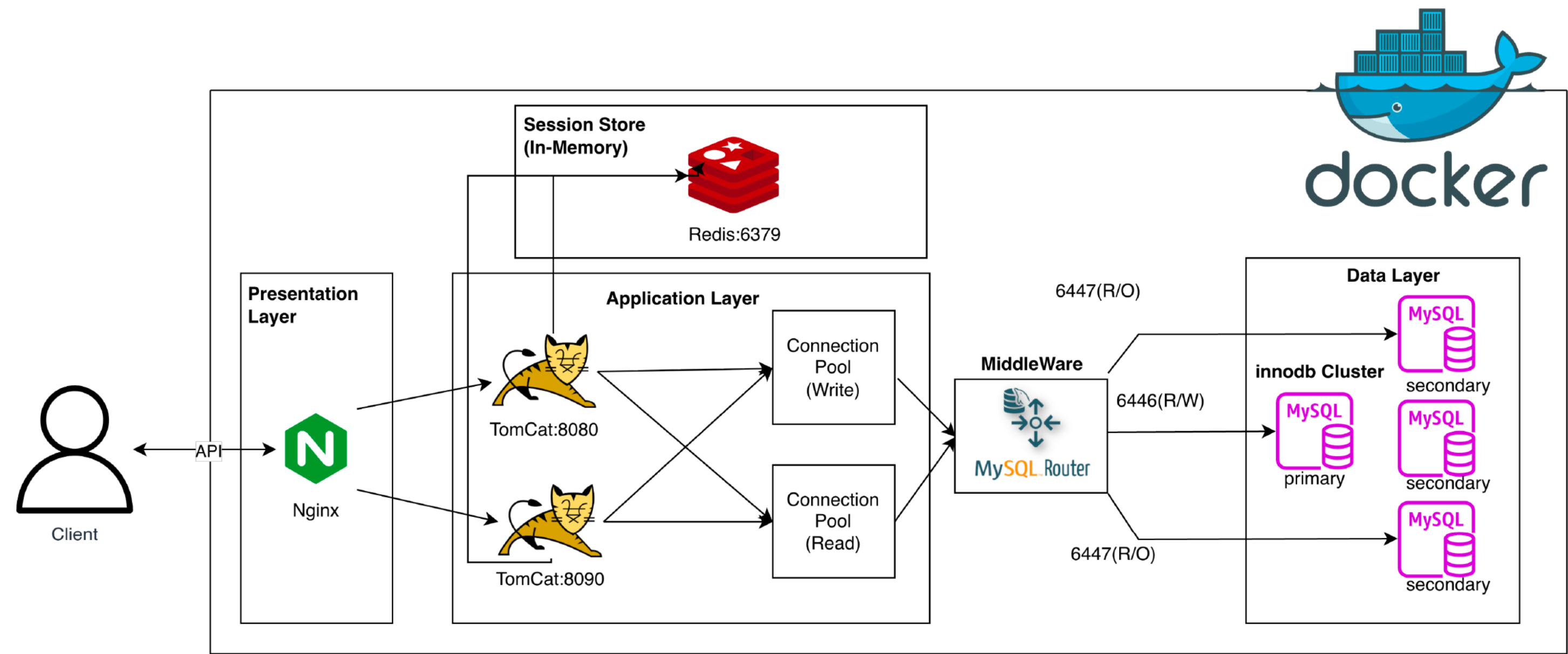
- 대기로 인한 점유를 막기위해 JPA의 @QueryHint의 lock.timeout을 0으로 설정하여 MySQL 8.x 환경에서 NOWAIT 옵션 사용 (즉시 종료)
- 근본적인 처리량의 천장을 늘리기 위해 Data Layer단에서 샤딩 혹은 원장 분리를 해야된다는 점을 알게되었습니다.
- 처리량을 보완하기 위해 DB 리소스를 Scale Up하여 JDBC Connection Pool의 기본값을 확대해야된다는 점을 알게되었습니다.
- 실무자에게 결제는 250 ~ 300 TPS가 나와야 정상이라는 목표 기준도 받아 향 후 리팩토링 방향을 잡았습니다.

결과

- "제한을 걸면 빨라진다"는 통념을 부하 데이터로 반증하였습니다.
- 진짜 병목은 단일 행에 대한 비관적 락과 DB 커넥션 풀 (10개)이라는 것을 47req/s라는 수치로 규명했습니다.
- Rate Limiting의 정상 동작을 확인했고, Grafana로 적용 전 → 적용 후의 차이를 눈으로 확인했습니다.



Project – Woori Card Scope



서비스

우리카드의 538만 건 고객 데이터를 기반으로 카드 이용 현황을 조회·분석하는 웹 애플리케이션입니다.
3-Tier 아키텍처 전 계층(WAS · DB · 세션)에 이중화를 적용하여 은행 시스템에 요구되는 고가용성을 구현했습니다.

작업 기간

2025. 02. 27 - 2026.03. 04 (1 주일)

기술 스택

-  Java 21
-  Tomcat 9
-  MySQL 8.0 (InnoDB Cluster)
-  Redis 7 (Redisson Tomcat Session Manager)

작업 내용

대용량 데이터 조회시 병목현상 개선 및 실행계획 분석 후 불필요한 인덱스 제거
DB Master - Replica 문제점 개선 및 Errant GTID 문제 해결

Trouble Shooting

1. Deffered Join - 대용량 조회 성능과 커버링 인덱스

조회 방식	실행시간	원래 방식 대비
집계 후 OFFSET (원래 방식)	38,890.0 ms	기준
Deferred Join · idx_seq 없음	14,844.9 ms	2.6배 빠름
Deferred Join + idx_seq	1,857.9 ms	20.9배 빠름

문제

- 카드 거래내역 테이블 안에는 고객·분기별 소비 내역으로, 대·중·소분류 금액 컬럼만 50여 개인 넓은 행 구조입니다. (538만 행)
- 목록을 뒤 페이지로 넘길수록 조회가 급격히 느려졌습니다. 실측하니 첫 페이지조차 14,526.6 ms가 걸렸습니다.

원인 분석

- LIMIT 20 OFFSET N은 건너뛸 N개 행을 전부 읽고 버려, 뒤 페이지일수록 비용이 커집니다.
- 더 근본적으로, 운영 테이블을 EXPLAIN으로 확인하니 인덱스가 하나도 없었습니다. (적재 시 DDL의 인덱스가 누락)
- 그래서 SELECT DISTINCT SEQ ... ORDER BY SEQ가 매번 538만 행을 전체 스캔 후 정렬하고 있었습니다.

해결

- Deferred Join: 인덱스만으로 페이지에 해당하는 PK(SEQ)를 먼저 추린 뒤, 그 PK로만 본문을 조회합니다.
→ JOIN (SELECT DISTINCT SEQ FROM card_transaction ORDER BY SEQ LIMIT 20 OFFSET ?) tmp ON c.SEQ = tmp.SEQ
- 커버링 인덱스: 이 방식이 성립하려면 좁은 인덱스가 필요하므로 SEQ 단일 커버링 인덱스를 추가했습니다.
→ CREATE INDEX idx_seq ON card_transaction (SEQ);
- EXPLAIN ANALYZE로 실제 동작을 확인했습니다.

```
-> Sort: c.SEQ (actual time=1814..1814 rows=34 loops=1)
-> Table scan on <temporary> (actual time=1814..1814 rows=34 loops=1)
-> Aggregate using temporary table (actual time=1814..1814 rows=34 loops=1)
-> Nested loop inner join (cost=5.16e+6 rows=4.61e+6) (actual time=1802..1814 rows=117 loops=1)
-> Filter: (tmp.SEQ is not null) (actual time=1801..1801 rows=20 loops=1)
-> Table scan on tmp (actual time=1801..1801 rows=20 loops=1)
-> Materialize (actual time=1801..1801 rows=20 loops=1)
-> Limit/Offset: 20/800000 row(s) (actual time=1801..1801 rows=20 loops=1)
-> Covering index skip scan for deduplication on card_transaction using idx_seq (actual time=1.02..1787 rows=800020 loops=1)
-> Index lookup on c using idx_seq (SEQ=tmp.SEQ) (actual time=0.307..0.636 rows=5.85 loops=20)
```

· 초록: 서브쿼리가 idx_seq 만으로 처리(커버링 인덱스) — 넓은 본문 행을 읽지 않고 SEQ만 훑어 800,020건을 dedup.

· 노랑: 추려진 20개 SEQ만 본문 JOIN(loops=20 , SEQ당 평균 5.85행) — 페이지 깊이와 무관하게 본문 접근은 페이지 크기로 고정.

· 상위 Aggregate / Sort 도 이 소수 행에만 적용 → naive처럼 538만 행 전체를 집계·정렬하지 않음.

- 이어서 필터 검색(등급, 연령, 성별, 지역)용 복합 인덱스(idx_filter)도 추가했지만, EXPLAIN을 찍어보니 옵티마이저가 사용하지 않았습니다.

```
-> Sort: c.SEQ (actual time=5.85..5.85 rows=31 loops=1)
-> Table scan on <temporary> (actual time=5.83..5.83 rows=31 loops=1)
-> Aggregate using temporary table (actual time=5.83..5.83 rows=31 loops=1)
-> Nested loop inner join (actual time=5.66..5.78 rows=71 loops=1)
-> Filter: (tmp.SEQ is not null) (actual time=5.65..5.65 rows=20 loops=1)
-> Table scan on tmp (actual time=5.65..5.65 rows=20 loops=1)
-> Materialize (actual time=5.65..5.65 rows=20 loops=1)
-> Limit: 20 row(s) (actual time=0.492..5.64 rows=20 loops=1)
-> Group (no aggregates) (actual time=0.492..5.64 rows=20 loops=1)
-> Filter: (SEX_CD=@sex and AGE=@age and MBR_RK=@rk) (actual time=0.151..5.63 rows=72 loops=1)
-> Index scan on card_transaction using idx_seq (actual time=0.009..5.47 rows=2136 loops=1)
-> Index lookup on c using idx_seq (SEQ=tmp.SEQ) (actual time=0.307..0.636 rows=5.05 loops=20)
```

· 초록: 필터 검색인데도 SEQ 순서 인덱스(idx_seq)를 스캔합니다(2,136행). idx_filter는 계획에 없음.

· 노랑: 등급·연령·성별 조건은 인덱스 스캔 위에 Filter로 얹혀 처리 → 20건을 채우는 순간 종료.

- 커버링 인덱스로 훑다 20건 채우면 멈추는 게(2,136), 조건에 맞는 46,738명을 전부 찾아 SEQ로 정렬 후 자르는 복합 인덱스보다 싸다고 판단했습니다.

결과

- Deferred Join (o) + Index (x): 14,844.9 ms → 1,857.9 ms, 기존 시간보다 87.48% 감소했습니다. (8배)
- 깊은 페이지 (OFFSET 800,000): 38,890 ms → 1,857.9 ms, 기존 시간보다 95.22% 감소했습니다 (20.9배)
- 첫 페이지 (OFFSET 0): 14,527 ms → 2.3 ms, 기존 시간보다 99.98% 감소했습니다. (6,313배)
- 배운점: 실행 계획을 확인하며 그에 맞는 방식을 적용할 수 있었습니다. (인덱스는 만든다고 쓰이는 게 아니었습니다.)
- 한계점: 커버링 인덱스를 적용해도 OFFSET만큼은 인덱스로 훑으므로, 깊은 페이지에 대한 비용은 어쩔 수 없었습니다.

2. InnoDB Cluster - Master-Replica의 split-brain 방지

2-1. 왜 단순 Master-Replica가 아니라 InnoDB Cluster인가

문제

- 단일 DB는 죽으면 서비스 전체가 멈춥니다. 따라서 이중화가 필요합니다.
- 하지만 단순 Master - Replica (비동기 복제)에는 문제가 있습니다. Master가 죽어 Replica를 새 Master로 승격시킨 뒤 죽었던 옛 Master가 살아나면?
 - 두 노드가 모두 Master로 행세하는 split - brain이 생길 수 있습니다.
 - 양쪽에서 쓰기가 발생하면 데이터 무결성이 깨지게 되고, 되돌리기 어렵습니다.

원인 분석

- 비동기 복제는 데이터를 따라 쓸 뿐 "누가 Primary인지"를 시스템이 합의로 정하지 않습니다.
- 승격·복귀 판단이 사람 손에 있으면 split-brain을 막을 안전장치가 없습니다.
 - ① 과반수 합의로 Primary를 하나만 정합니다.
 - ② 되살아난 노드가 제멋대로 Master가 되지 못하게 이력을 정합시키는 구조가 필요합니다.

해결

- 총 4개의 노드 InnoDB Cluster (Single - Primary)의 Group Replication을 활용하여 커밋/멤버 변경을 과반수 합의로 확정시킵니다.
 - 어느 순간에도 Primary는 하나뿐이게 막았습니다. (split brain 차단)
- 앞단에 MySQL Router(6446 R/W→Primary, 6447 R/O→Secondary 분산)를 사용하여, WAS는 Router에만 연결합니다.
- HikariCP도 write / read pool을 나눠서 조회 전용 DAO에선 read-pool만 사용합니다.
- 4 노드인 이유: **quorum = $\lfloor N/2 \rfloor + 1$. N=4 → 3**, 즉 1노드 장애까지 **자동 복구**
- 참고: 노드의 개수는 홀수가 정석이고 노드 4개와 3개는 내구성이 같습니다.

3-2. 노드 합류 실패 시 Errant GTID 발생과 Clone 복구 과정

문제

- 클러스터를 구성하기 위해 seed(mysql1)에 나머지 노드를 addInstance로 추가했지만, mysql2·3·4에 각각 Errant GTID가 존재해 표준 방식으로는 클러스터에 합류할 수 없었습니다.
 - mysql2:3306 has the following errant GTIDs that do not exist in the cluster: ade01245-aa79-11f1-a124-7e0235ae1376:1-18
- 세 노드 모두 서로 다른 UUID로 :1-18의 트랜잭션을 갖고 있었습니다. 즉, 클러스터를 구성하기 전부터 네 노드의 트랜잭션 이력이 서로 달랐던 것입니다.

원인 분석

- GTID는 각 노드의 트랜잭션 이력을 비교하는 기준이며, Errant GTID는 그룹에 없는 트랜잭션이 합류 노드에만 존재해 이력이 발산한 상태를 의미합니다.
- 근본 원인은 클러스터 형성 전에 각 노드가 독립적으로 초기화된 설정 구조였습니다.
 - **docker-compose가 init 스크립트를 4개 노드 전부에 마운트**
 - **my.cnf가 컨테이너 기동 시점부터 GTID·바이너리 로그를 활성화**
- 이 조합으로 초기화 SQL이 각 노드에서 개별 실행되면서, 동일한 스키마라도 노드별 server_uuid 기반의 서로 다른 GTID가 기록됐습니다.
- 즉 클러스터를 만들기 전에 이미 노드마다 다른 트랜잭션 이력이 쌓여 있었습니다.

해결

- clone 기반 복구로 변경했습니다.
 - **cluster.addInstance('root@mysql2:3306',{ recoveryMethod: 'clone' });**
- clone은 합류 노드의 데이터, GTID를 전부 버리고 donor의 물리 스냅샷을 통째로 복사합니다. 잉여 이력을 지우고 seed의 이력으로 덮어씹습니다.

```
** Stage DROP DATA: Completed           # 기존 데이터·GTID 폐기
** Stage FILE COPY / PAGE COPY: Completed
* Clone process has finished: 80.64 MB transferred in about 1 second
```

- binlog 복구도 검토했으나, 노드마다 트랜잭션 이력이 이미 서로 다르게 갈라진 상태에서는 로그 재생만으로 두 히스토리를 하나로 합칠 수 없다고 판단해 기각했습니다.

결과

- 자동 Failover 측정 결과:

시나리오	장애 주입	결과
하드 크래시	<code>docker kill mysql1</code>	mysql2 자동 승격, R/W 복구 22.15초
정상 종료	<code>docker stop mysql2</code>	mysql1 재승격, R/W 복구 21.91초

Project - 스뮤클럽

SMU CLUB

상명대학교 동아리 통합 플랫폼



서비스

<https://github.com/smu-human/smu-club>

대학교 동아리 지원&관리 플랫폼

작업 기간

2025. 08. 15 - (1년 이상 운영중)

백엔드 기술 스택

 Spring

 MySQL

구성원

차준규 - BE

유승준 - BE

이윤표 - FE

작업 내용

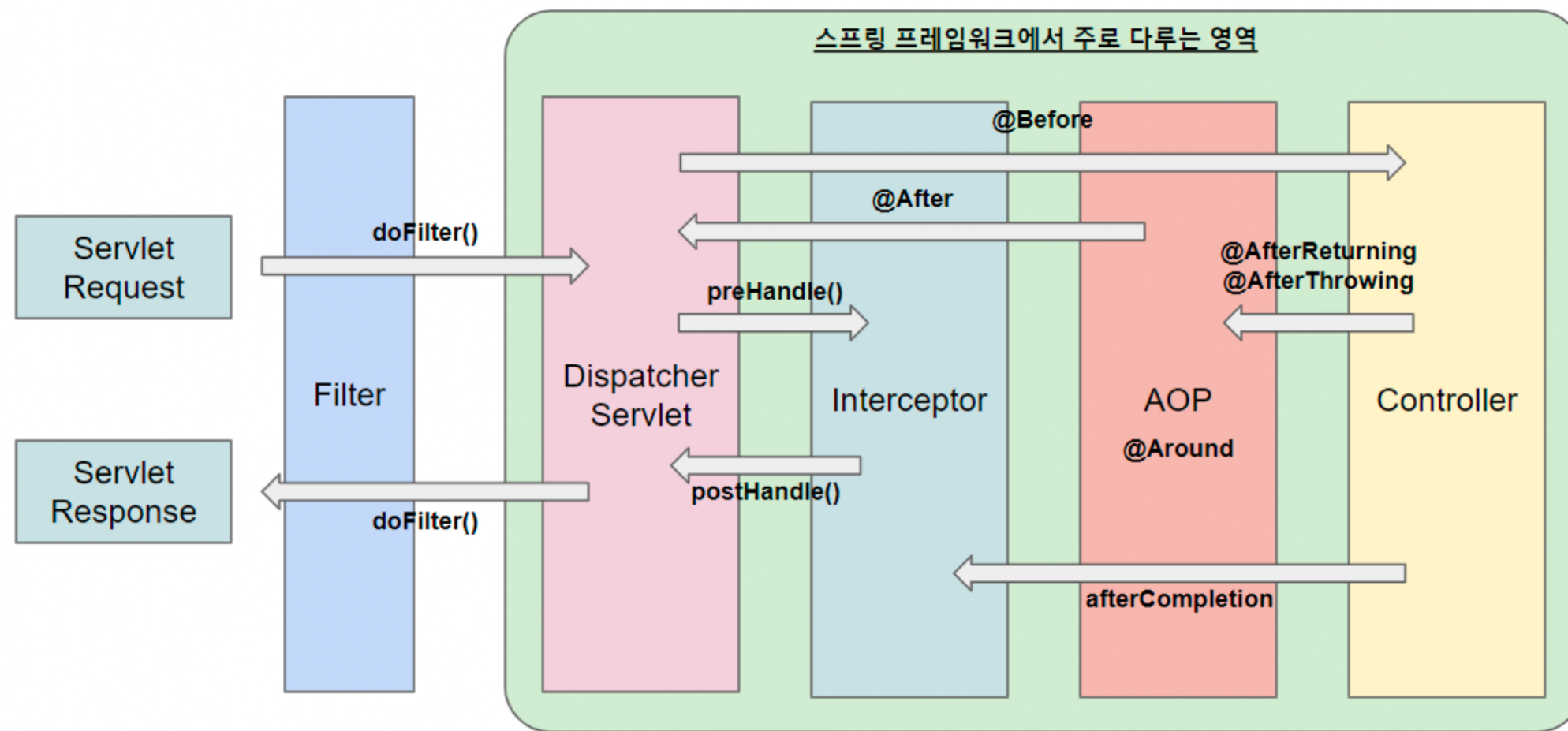
Dispatcher Servlet preHandle()를 이용한 Session 토큰 발급 및 검증

배치성 작업의 트랜잭션 범위를 재설계해 실패 전파 범위 축소

메일 발송을 비동기 처리로 전환해 서버 블로킹을 해소

Trouble Shooting

1. 로그인 기능을 걷어내고 비로그인 인증을 다시 설계한 과정



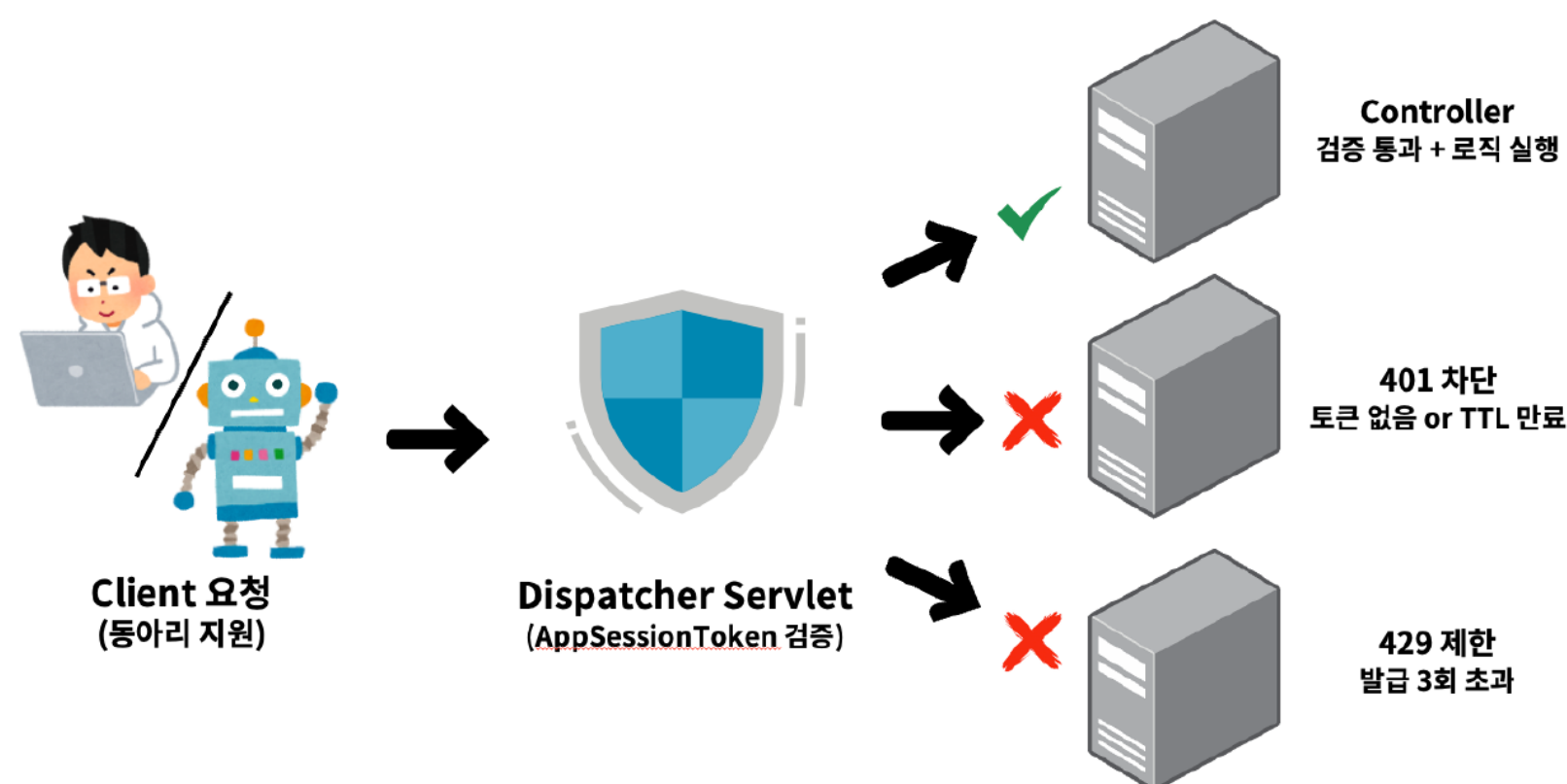
문제

- 초기 프로세스는 학교 SSO로 로그인해야 지원할 수 있는 구조였고, 로그인 계정 정보를 서비스가 넘겨받아야 했습니다.
- 학교 이해관계자와 법률 자문을 진행한 결과, 서비스가 학교 계정 인증을 대신 처리하는 것이 학교 측에 제도적 책임을 발생시킨다는 점을 확인했습니다.
- 이미 만들어 둔 기능이었지만, 편의보다 책임 소재가 우선이라 판단해 SSO 로그인을 걷어내기로 했습니다.

원인 분석

- 로그인을 없애면 "요청을 보낸 사람이 조금 전 지원서를 작성하던 그 사용자인지"를 확인할 근거가 사라집니다.
- 인증이 사라진 자리에서는 지원과 무관한 외부 요청이나 파일 업로드용 presigned URL 발급을 무제한으로 시도하는 일을 막을 수 없었습니다.
- 즉 로그인 제거는 단순히 화면 하나를 빼는 일이 아니라, 로그인이 담당하던 "요청 주체 식별"을 대신할 장치가 필요한 문제였습니다.

해결



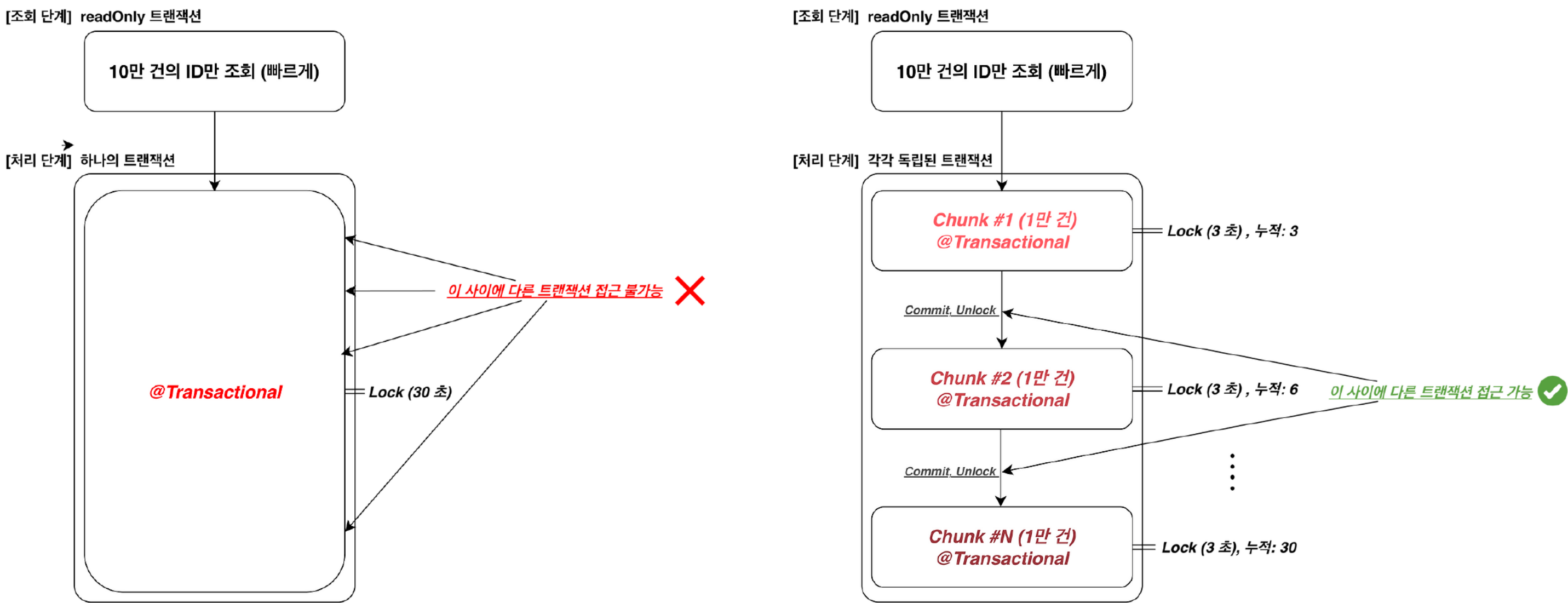
- 로그인 대신, 지원 흐름에 진입하는 시점에 서버가 임시 세션 토큰(AppSessionToken)을 발급하도록 설계했습니다. (UUID 값에 30분 TTL 부여)
- 모든 요청이 아니라 지원과 관련된 메서드에만 인증을 적용하기 위해, @RequiresAppSession 커스텀 어노테이션을 만들고 Spring Interceptor의 preHandle()에서 토큰을 검증했습니다.
- 토큰이 없거나 TTL이 만료된 요청은 401로 차단하고, presigned URL 발급을 한 세션에서 3회 넘게 시도하면 429로 제한했습니다.
- 검증을 통과한 토큰은 request.setAttribute()로 Controller에 넘겨 이후 로직에서 재사용하도록 했습니다.
- IP 단위 제한도 검토했지만, 학교·기숙사처럼 공유 IP(NAT) 환경에서는 한 사람의 과다 요청이 같은 경우가 발생하여, 공정하지 않다고 판단했습니다.

결과

- 로그인 없이도 지원자가 신원 노출 없이 지원할 수 있게 되면서, 학교에 인증 책임을 지우지 않는 구조로 전환했습니다.
- 무제한 presigned URL 발급 시도와 지원과 무관한 요청을, 토큰 유무와 발급 횟수만으로 걸러낼 수 있게 되었습니다.
- 이미 만든 기능이라도 책임 소재가 확인되면 걷어내고 다시 설계하는 것이, 눈앞의 편의를 지키는 것보다 낫다는 판단 기준을 세웠습니다.

- 자세한 내용을 확인하고 싶다면, 해당 블로그 글을 참조해주세요.

2. 배치성 작업을 단일 트랜잭션으로 처리하게 될 경우 롤백 문제



문제

- 모집 자동 마감, 이메일 재전송과 같은 배치 작업을 하나의 큰 트랜잭션으로 처리하고 있었습니다.
- 일부 데이터 처리에 실패하면 전체 작업이 롤백되어 이미 처리된 데이터까지 다시 수행해야 했습니다.
- 또한 트랜잭션이 종료될 때까지 Lock을 장시간 점유하여 운영 중 다른 트랜잭션과 충돌할 가능성이 있었습니다.

원인 분석

- 배치 전체를 하나의 트랜잭션으로 처리하여 Lock 유지 시간이 전체 처리 시간만큼 길어졌습니다.
- 실패 지점을 정확히 알기 어려워 장애 분석과 재처리 범위가 불필요하게 커졌습니다.
- 조회와 변경 작업이 동일한 트랜잭션에서 수행되어 메모리 사용량과 트랜잭션 부담도 증가했습니다.

해결

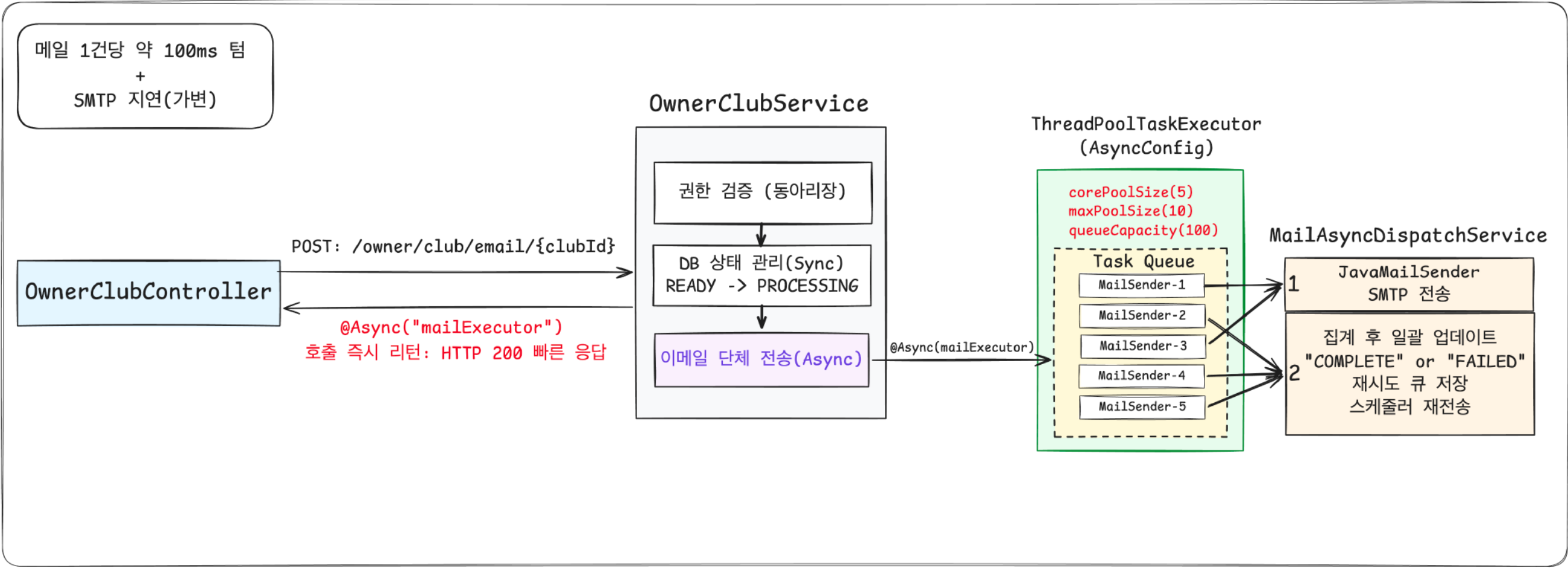
- 배치 특성에 맞게 조회와 변경 작업의 트랜잭션 범위를 분리하도록 구조를 재설계했습니다.
- 모집 자동 마감은 종료 대상만 조회한 뒤 10 단위로 Chunk를 분할하고, 각 Chunk를 REQUIRES_NEW 트랜잭션으로 독립 처리하도록 변경
- 이메일 재전송 역시 각 작업을 독립 트랜잭션으로 수행하여 성공 여부를 개별적으로 관리하고, 실패 시에는 백오프와 재시도를 적용하도록 구현
- 이를 통해 Chunk 단위로 Commit과 Lock 해제가 이루어져 다른 트랜잭션이 중간에 수행될 수 있도록 개선했습니다.

결과

- 일부 데이터 처리 실패가 전체 배치를 무효화하지 않도록 개선했습니다.
- Lock 점유 시간을 전체 작업 기준 약 30초에서 Chunk당 약 3초 수준으로 분산하여 운영 중 충돌 가능성을 줄였습니다.
- 실패 지점을 Chunk 단위로 추적할 수 있어 장애 분석과 재처리 범위가 명확해졌습니다.
- 전체 엔티티를 유지하지 않고 필요한 데이터만 처리하도록 변경하여 메모리 사용량도 함께 감소했습니다.

항목	Before	After
트랜잭션 수	1개 (전체)	N개 (Chunk 개수만큼)
Lock 유지 시간	전체 처리 시간	Chunk당 처리 시간 (1/10 이하)
실패 시 영향	전체 롤백	Chunk 단위 롤백
사용자 영향	ConnectionTimeout 위험	최소화
메모리 사용량	전체 엔티티 로드	필요한 컬럼만 조회

3. 단체 메일을 동기 방식으로 발송하면, 완료될 때까지 블로킹되던 문제



문제

- 관리자가 합불 결과 메일 발송 버튼을 누르면 지원자 전체에게 개별 메일을 발송해야 했습니다.
- 초기 구현은 for문으로 메일을 순차적으로 발송하는 동기 방식이었습니다.
- SMTP I/O 특성상 메일 1건당 평균 1초가 소요되어, 100명 기준 약 100초 동안 메인 스레드가 블로킹되었습니다.
- 그 결과 관리자는 화면이 멈춘 상태로 대기해야 했고, 장시간 요청으로 인해 504 Gateway Timeout이 발생했습니다.

원인 분석

- 메일 발송을 요청을 처리하는 동일한 스레드에서 수행하여 HTTP 요청이 종료되지 않았습니다.
- SMTP는 I/O Bound 작업임에도 병렬 처리가 이루어지지 않아 처리 시간이 발송 건수에 비례하여 증가했습니다.
- 단순히 @Async만 사용할 경우 Spring의 기본 SimpleAsyncTaskExecutor는 요청마다 새로운 스레드를 생성하므로, 대량 발송 시 스레드가 무한히 증가해 서버 안정성을 해칠 수 있었습니다.

해결

- 메일 발송을 @Async 기반 비동기 작업으로 변경하여 메인 스레드는 즉시 "메일 전송이 시작되었습니다."를 반환하도록 개선했습니다.
- ThreadPoolTaskExecutor(corePoolSize=5, maxPoolSize=10, queueCapacity=100)를 구성하여 스레드 수를 제한하고 안정적으로 병렬 처리하도록 설계했습니다.
- 각 메일 발송 메서드는 CompletableFuture<Void>를 반환하도록 구현하고, 모든 작업을 CompletableFuture.allOf()로 수집하여 전체 발송 완료 시점을 확인할 수 있도록 했습니다.
- 개별 메일 발송 실패는 CompletableFuture 내부에서 예외를 처리하여 다른 메일 발송에는 영향을 주지 않도록 구성했습니다.

결과

- 관리자는 버튼 클릭 후 약 0.1초 내에 응답을 받아 504 Gateway Timeout 문제가 해결되었습니다.
- 동일한 100건 발송 기준에서 93.8초 → 23.0초로 총 처리 시간이 75.4% 단축되었으며, 처리량은 1.07건/초 → 4.34건/초(약 4.1배) 향상
- ThreadPoolTaskExecutor를 통해 스레드 수를 제한하여 대량 메일 발송 시에도 서버 안정성을 유지
- CompletableFuture.allOf()를 통해 전체 발송 완료 여부를 확인할 수 있게 되었으며, 개별 메일 실패는 다른 작업과 격리되어 나머지 메일은 정상적으로 발송되었습니다.

```
backend-1 | [SYNC-BENCH] 동기 메일 배치 종료
backend-1 | - clubId: 1
backend-1 | - attempted: 100
backend-1 | - success: 100
backend-1 | - failed: 0
backend-1 | - retryQueued: 0
backend-1 | - elapsedMs: 93765
backend-1 | - throughputEmailPerSec: 1.07
```



```
backend-1 | [ASYNC] 비 동기 메일 배치 종료
backend-1 | - clubId: 1
backend-1 | - attempted: 100
backend-1 | - success: 100
backend-1 | - failed: 0
backend-1 | - retryQueued: 0
backend-1 | - elapsedMs: 23039
backend-1 | - throughputEmailPerSec: 4.34
```